

Analog AI Accelerators for Transformer-based Language Models: Hardware, Workload, and Power Performance

H. Tsai^{*1}, H. Benmeziane², I. Boybat², J. Büchel², P. Narayanan¹, M. Le Gallo², S. Jain³,
A. Vasilopoulos², W. Simon², K. Hosokawa⁴, M. Ishii⁴, Y. Kohda⁴, A. Chen¹, C. Mackin¹,
K. El Maghraoui³, A. Okazaki⁴, A. M. Friz¹, J. Luquin¹, A. Sebastian², V. Narayanan³, and G. W. Burr¹

¹ IBM Research—Almaden, San Jose, CA, USA

³ IBM Research—T. J. Watson Research Center, Yorktown Heights, NY, USA

² IBM Research—Zurich, Rüschlikon, Switzerland

⁴ IBM Tokyo Research Laboratory, Tokyo, Japan.

*email: htsai@us.ibm.com

Abstract—Transformer-based Large Language Models (LLMs) demand large weight capacity, efficient computing, and high throughput access to large amount of dynamic memory. These challenges present great opportunities for algorithmic and hardware innovations, including Analog AI accelerators. In this paper, we describe recent progress on Phase Change Memory-based hardware and architectural designs to address the challenges for LLM inference.

Index Terms—Analog In-Memory Computing, Phase Change Memory, 3D AIMC.

I. INTRODUCTION

Transformer-based language models have gained significant traction in real-world artificial intelligence (AI) applications since the inception of the “Transformer” attention-based network architecture [1]. These models have evolved over time from *encoder-only* architectures for language embedding and language classification, to *encoder-decoder* architectures for machine translation and question answering, to the *decoder-only* architecture predominantly used for Large Language Models (LLMs) [2].

As with earlier types of Deep Neural Network (DNN) models, empirical evidence has shown that larger Transformer models, with wider and/or more layers, achieve higher accuracy across different types of benchmarking tasks, such as language modeling, reasoning, and more. As a result, the number of Transformer model parameters has been growing at 8x every 2 years, far outpacing both hardware memory capacity growth and Moore’s Law. In order to meet the growing compute demands, these massive DNN workloads are often split across a large number of workers residing on different chips, cards, or even data-center racks. All this scale-out implies yet more energy dedicated to data communication, on top of the memory-to-CPU communication required by the Von Neumann architecture, rather than to actual compute. Another challenge associated with LLM workloads is the fast-growing demand for longer sequence- or context-lengths, from hundreds to many thousands if not millions of language tokens.

Several algorithmic improvements have been developed to address these performance bottlenecks, especially during model inference. Mixture of Expert (MoE) models [3] replace single, large, fully-connected weight-layers with multiple smaller “experts,” allowing a router to select only a subset of active weights during inference time. This allows the

models to contain a large number of weights and thus offer a wide range of capabilities, while limiting the hardware costs during inference. Alternative network architectures to the traditional Multi-Head Attention (MHA) are also actively being explored to reduce the hardware costs of attention compute. Grouped Query Attention (GQA), used by most large Llama models [4], shares a single Key and Value vector across a group of attention-heads (each with its own Query vector), which reduces the size of the KV-cache that must be saved for subsequent token generation. Multi-head Latent Attention (MLA), used by Deepseek-v3, utilizes compression to further reduce KV-cache sizes [5]. State-space models (SSMs), such as Mamba [6] or Bamba [7], replace all or most MHA layers with recurrent layers to collapse one sequence dimension into a fixed size vector, resulting in savings for compute time, on-chip memory, and KV-cache.

II. ANALOG IN-MEMORY COMPUTE (AIMC)

Another path to alleviate the growing data communication for LLMs is to apply Analog In-Memory Compute (AIMC) to perform matrix-vector multiplication (MVM) at the location of the weights. AIMC maps DNN weights onto memory device conductance or charge and takes advantage of Ohm’s Law and Kirchhoff’s Law to compute MVMs at the location of the weights in an analog fashion [8]. The main advantages of AIMC lie in the high-density model-weight storage that leads to high area efficiency ($TOPS/mm^2$) and the more energy-efficient multiply and accumulate in analog ($TOPS/W$). AIMC can be implemented in various ways, for example, using volatile memory [9] or non-volatile memory (NVM) for weight storage; storing one or multiple bits of weights per memory device; and reading memory arrays using analog read voltages [10] or a constant voltage with analog durations [11]. Among these implementation options, AIMC with NVM is best suited for keeping static DNN weights on-chip for extended period of time and hence eliminating data transfer for model weights. Several NVM has been used to demonstrate AIMC, including phase change memory (PCM) [12], [13], resistive random access memory (ReRAM) [10], magnetoresistive random access memory (MRAM) [14], electrochemical random access memory (ECRAM) [15], optical memory [16], NOR [17] and NAND [18] flash memory. Keeping the architecture “weight

stationary” requires careful spatial mapping of model weights and efficient activation communication between AIMC tiles to realize the full system-level power performance potential for AIMC [19].

In this paper, we describe recent progress on AIMC hardware and architectural designs to address the challenges for LLM inference. The rest of the paper is organized as follows:

- Section III describes PCM-based hardware demonstrations and an AIMC architecture based on 14nm designs.
- Section IV discusses three specific challenges to implementing LLMs in AIMC systems: model accuracy, architecture scale-up, and the self-attention bottleneck.

III. PCM-BASED AIMC HARDWARE AND ARCHITECTURE

Phase-change memory (PCM) has many attractive attributes for AIMC, including fast read speed, multi-level resistance values, good memory window, high endurance, and mature integration in CMOS back-end-of-line (BEOL). IBM recently demonstrated two chips and one architectural extension using PCM-base AIMC on 14nm CMOS [12], [13], [19], as summarized in Table I.

TABLE I
14NM AIMC CHIP DEMONSTRATION AND ARCHITECTURE STUDY

Chip Attributes	Chip 1 [12]	Chip 2 [13]	Architecture [19]
Chip Area	$\sim 100mm^2$		
Tile-size	256×256	512×512	512×512
Tiles have ADC?	Yes	No	Yes
Tile-to-Tile communication	Dedicated Links	2D Mesh	2D Mesh
Num. of Tiles per Chip	64	34	48
Num. of Weights per Chip	4M	8.5M	12M
Energy Eff. (TOPS/W)	9.76 (peak) (measured)	12.4 (best chip) (measured)	100 (peak) (projected)
DNN Demo	FC, CNN, LSTM (including off-chip ops)		CNN, LSTM, Transformer

The first chip [12] includes 64 AIMC tiles and global digital compute units at the center of the chip for complex operations, such as activation functions and operations for Long-Short-Term Memory (LSTM). The tiles consist of 256×256 logical arrays with Current-Controlled Oscillator (CCO)-based ADCs and local digital compute at each tile for scaling and ReLU operations. The smaller tile size offers flexibility and high array utilization for mapping convolutional neural networks (CNNs) and smaller DNNs. In addition to demonstrating image classification and image captioning, the chip was also used to verify AIMC accuracies of base-calling DNNs for genome sequencing [20].

The second chip [13] includes 34 AIMC tiles with a massively parallel 2D-mesh for inter-tile communication using analog duration. Each tile consists of a 512×512 logical array and generates 512 analog durations as outputs that can be transmitted in parallel and directly used as inputs for the next tile. DNNs with only fully-connected (FC) layers and ReLU activations, *e.g.* a keyword spotting network, were fully implemented on chip in the analog domain. For DNNs with complex digital operations, analog durations were converted to

digital at the edge of the chip for computation, then sent back on chip for the next MAC operations. Using this methodology, a recurrent neural network transducer (RNNT) with 45 million weights was demonstrated at low Word-Error-Rate using 5 chips and about 140 million PCM devices, demonstrating the scalability of AIMC.

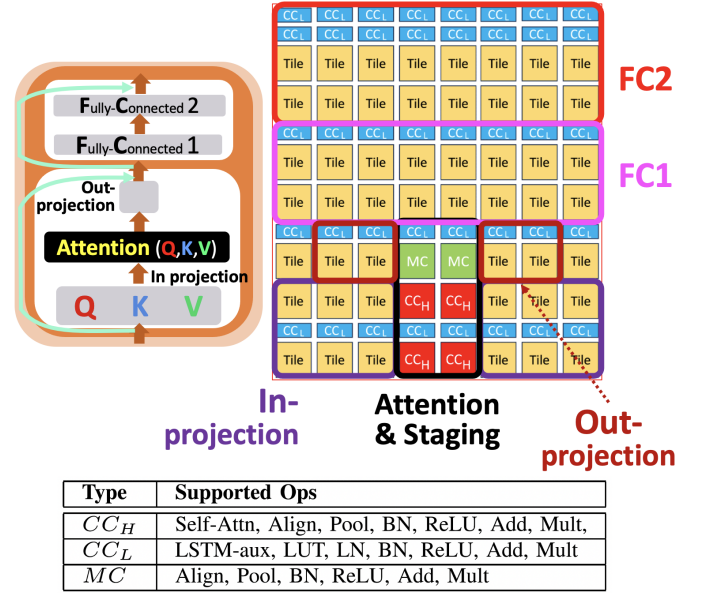


Fig. 1. AIMC architecture and example Transformer mapping for one layer of BERT-Large [21]. The Tile sizes are 512 by 512 and the specialized Heavy Compute Core (CC_H), Light Compute Core (CC_L), and Memory Core (MC) are designed and simulated in 14nm.

Combining insights from the 14nm hardware, we proposed a heterogeneous, programmable, and scalable system architecture with on-chip auxiliary operation support for CNN, LSTM, and Transformer [19]. Figure 1 shows an example of mapping one BERT Large layer onto the architecture [21], with projected 40-140x energy efficiency benefits compared to A100 GPUs. A similar analog fabric was designed for edge Transformer applications [22], hosting 33 million PCM weights on a $30mm^2$ chip. Architectural simulations on MobileBERT demonstrated throughput approaching some high-end SoCs for phones.

IV. AIMC FOR TRANSFORMER-BASED LLMs

The first challenge in using AIMC for LLMs is to confirm that high DNN model accuracies can be maintained despite the presence of AIMC non-idealities such as noise and conductance drift. To study networks well beyond the scale of current hardware implementations, the AIHWKit [23] and AIHWKit-Lightning [24] were developed to model AIMC for DNNs and enable hardware-aware (HWA) training [25]. AIHWKit-Lightning is a simplified and accelerated version to support HWA training of LLMs. A RoBERTa model that included HWA noise during pre-training demonstrated iso-accuracy on the GLUE benchmark [24]. To study the impact of post-training optimization techniques, we used the full AIHWKit to study the accuracy and noise-resilience of OPT, Llama, and Mistral models ranging from 2.7-13B weights [26].

The second challenge is to scale up the AIMC architecture to handle the large number of LLM weights. Volatile memory-based AIMC can potentially utilize wafer-scale technologies [27] for extreme capacity scaling. Larger tile sizes usually leads to higher memory array density and energy efficiency, but will be limited by IR-drop considerations as the long bitlines accumulate more and more read-current.

NVM technology has great potential for density scaling to host as many weights on one chip as possible. NVM-based AIMC has an additional knob to scale up memory capacity in 3D. Unlike in a fully 2D system, where each tile can have its dedicated peripheral circuitry, multiple tiers in a 3D tile can share input and output periphery, therefore allowing MAC to be computed in a One-Tier-at-a-Time (OTT) fashion, as shown in Figure 2. This makes the 3D AIMC system Fully Weight Stationary (FWS), and opens up the prospect for a “Fully Unrolled” system in which each layer has its own set of Tiles, allowing the entire workload to be fully-pipelined [21]. Compared to a Partially Weight Stationary (PWS) system, where weights are constantly reloaded from DRAM or SRAM, 3D AIMC maintains the more compact chip area without incurring the energy and latency for weight reloading. The 3D architecture is especially suitable for MoE models. By mapping experts in an MoE to different tiers of the same tile, all the experts remain weight-stationary, and data-vectors can be predictably delivered to the same Tile independent of which expert they need to address [28].

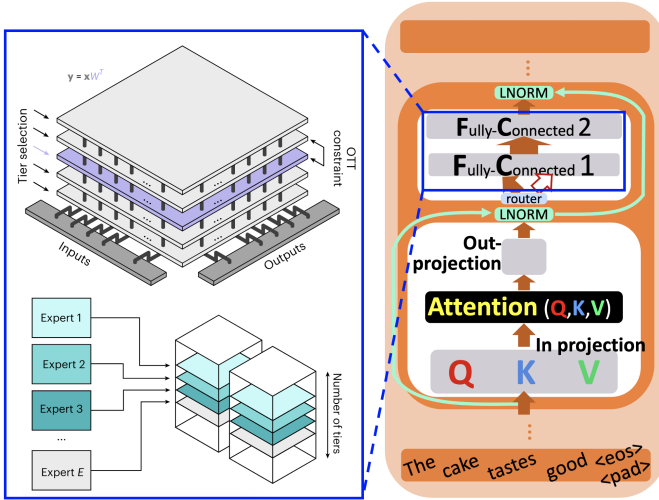


Fig. 2. Illustration for mapping Mixture of Expert (MoE) models onto a 3D AIMC [28]. The router directs each token to a handful of “experts,” each consisting of a separate fully-connected layer. By mapping experts into different tiers of a 3D AIMC and performing MAC operations One-Tier-at-a-Time (OTT), the weight capacity is fully utilized with high tile compute utilization and minimal increases in data-transport.

The third challenge is the self-attention bottleneck – the scope of the self-attention operations increases rapidly with sequence-length, s , of the Transformer model. A standard self-attention matrix requires computation of the inner product between the Query and Key vectors of every sequence token, followed by computations using the Value vectors of every sequence token. As a result, for an encoder-base model or

the pre-fill phase of a decoder-based model, attention compute scales as $O(s^2)$, while the other matrix multiplication compute on static weights scales as $O(s)$.

For the token-generation phase of a decoder-base model, attention compute scales as $O(s)$, but requires access to the Key and Value vectors from all previously-computed sequence tokens, known as the KV-cache. When KV-cache size exceeds on-chip cache capacity, data needs to be loaded repeatedly from off-chip, creating a memory bottleneck. Worse yet, the arithmetic intensity – the number of compute operations performed per byte retrieved from memory – is very low, leading to inherent energy inefficiencies. In both pre-fill and generation phases, when sequence length s becomes excessively long, the attention part of the Transformer workload becomes a serious hardware bottleneck.

These issues apply to both GPU-like digital accelerators and AIMC, because self-attention matrices are computed in digital hardware even in an AIMC system. This is because self-attention compute involves MVMs between two dynamically computed matrices that are determined by the user input. Unlike the static weights, dynamic activations are not well-suited for NVM-based AIMC due to write speed and endurance limitations. While volatile-memory-based AIMC can potentially be used for self-attention compute, its limited memory capacity usually prevents static mapping of model weights, resulting in the requirement for very large chip area or model weight transfer from DRAM.

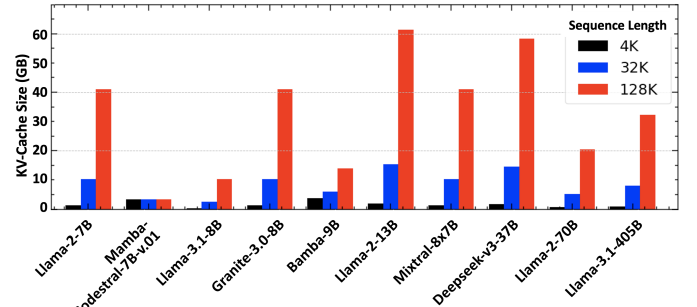


Fig. 3. KV-cache sizes when running LLMs on GPUs at a batch-size of 1 for sequence lengths of 4K, 32K, and 128K. The models are ordered based on their active model sizes during inference. Deepseek [5] and Mixtral [3] are Mixture of Expert (MoE) models, Mamba [6] is a pure State-Space Model (SSM), and Bamba [7] uses a mixture of SSM and multi-head attention.

The fundamental way to address the self-attention bottleneck is to change the LLM network architecture, switching to more efficient self-attention mechanisms that need less KV-cache. Because the KV-cache bottleneck also dominates GPU performance, AIMC can directly take advantage of the latest model improvements from the digital world. Figure 3 shows the experimentally measured KV-cache size when running the LLMs on GPUs. While longer sequence-lengths consistently lead to more KV-cache, newer LLMs use more efficient self-attention mechanisms, such as GQA and MLA, to limit KV-cache sizes despite the continued growth in model-size. The rapidly changing self-attention implementation helps improve LLM efficiency, but also creates challenges for designing application-specific ICs for self-attention. As a result, more

flexible designs, such as RISC-V based or other programmable compute cores, can be considered as a more future-proof option. Finally, having large on-chip SRAM buffers will be critical for storing KV-cache and reducing or hiding DRAM fetching as much as possible. One additional benefit for NVM-based AIMC is that since weight reloading is not required, there is no latency or energy benefit for running at large batch-sizes. Keeping batch-size low reduces the amount of activation storage and KV-cache required. As in most GPUs and digital accelerators, continued reduction in digital compute precisions for self-attention and other non-MVM operations can also significantly reduce digital compute and memory overhead in an AIMC system.

The architectural considerations and the corresponding technology and algorithm enablers for improving hardware power-and-performance are summarized in Table II.

TABLE II
AIMC DESIGN CONSIDERATIONS FOR LLM INFERENCE

System Components	Desired Attributes	Technology/Algorithm Enabler
Analog Tiles	High accuracy High weight capacity Energy efficiency Large tile sizes	Device, Design, HWA Memory scaling, 3D Low current, fast read IR-drop mitigation
Attention Compute Unit	Large SRAM High flexibility Reduce offchip data-fetch	CMOS scaling ASIC/Chiplet designs Reduce KV, low batch

V. CONCLUSION

NVM-based Analog In-Memory Compute (AIMC) is a promising candidate for serving Large Language Model (LLM) inference workloads. MAC-level and system-level AIMC has been demonstrated in hardware, up to the scale of tens of millions of model-weights. We address AIMC accuracy for LLMs by conducting hardware-aware pre-training and post-training optimizations using AIHWKit. Architecture analysis shows that AIMC scale-up to support LLMs requires technology scaling, memory density scaling, custom digital compute hardware, and improved LLM network architectures, especially for self-attention compute. Continued algorithm-hardware co-optimization will be key to improve hardware efficiency.

ACKNOWLEDGMENT

We thank V. Mukherjee, M. Furmanek, S. Munetoh, S. Yamamichi, J. Burns, M. Khare, the IBM Research AI Hardware Center, and the Albany NanoTech for management and logistical support.

REFERENCES

- [1] A. Vaswani *et al.*, "Attention is All you Need," in *Adv. Neur. Inf. Proc. Sys.*, pp. 5998–6008, 2017.
- [2] "Granite 3.0 language models," 2024. www.rivista.ai/wp-content/uploads/2024/10/paper-1.pdf.
- [3] A. Q. Jiang, A. Sablayrolles, A. Roux, *et al.*, "Mixtral of experts," *arXiv:2401.04088*, 2024.
- [4] J. Ainslie, J. Lee-Thorp, M. de Jong, *et al.*, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," *arXiv:2305.13245*, 2023.
- [5] A. Liu, B. Feng, B. Xue, *et al.*, "DeepSeek-V3 Technical Report," *arXiv:2412.19437*, Dec. 2024.
- [6] T. Dao and A. Gu, "Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality," *arXiv:2405.21060*, 2024.
- [7] "Bamba: Inference-efficient hybrid Mamba2 model," 2024. huggingface.co/blog/bamba.
- [8] G. W. Burr, A. Sebastian, T. Ando, and W. Haensch, "Ohm's law + Kirchhoff's current law = better AI: neural-network processing done in memory with analog circuits will save energy," *IEEE Spectrum*, vol. 58, no. 12, pp. 44–49, 2021.
- [9] C.-J. Jhang, C.-X. Xue, J.-M. Hung, *et al.*, "Challenges and trends of SRAM-based computing-in-memory for AI edge devices," *IEEE Trans. Circ. Sys.*, vol. 68, no. 5, pp. 1773–1786, 2021.
- [10] W. Wan, R. Kubendran, C. Schaefer, *et al.*, "A compute-in-memory chip based on resistive random-access memory," *Nature*, vol. 608, no. 7923, pp. 504–512, 2022.
- [11] P. Narayanan, S. Ambrogio, A. Okazaki, *et al.*, "Fully On-Chip MAC at 14 nm Enabled by Accurate Row-Wise Programming of PCM-Based Weights and Parallel Vector-Transport in Duration-Format," *IEEE Trans. Electr. Dev.*, vol. 68, no. 12, pp. 6629–6636, 2021.
- [12] M. Le Gallo, R. Khaddam-Aljameh, M. Stanisavljevic, *et al.*, "A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference," *Nature Electr.*, vol. 6, no. 9, pp. 680–693, 2023.
- [13] S. Ambrogio, P. Narayanan, A. Okazaki, *et al.*, "An analog-AI chip for energy-efficient speech recognition and transcription," *Nature*, vol. 620, pp. 768–775, Aug 2023.
- [14] S. Angizi, Z. He, A. Awad, and D. Fan, "MRIMA: an MRAM-based in-memory accelerator," *IEEE Trans. Computer Aided. Design Integr. Circ. & Sys.*, vol. 39, no. 5, pp. 1123–1136, 2019.
- [15] M. Um, M. Kang, H. Kwak, K. Noh, S. Kim, and H.-M. Lee, "An ECRAM-based analog compute-in-memory neuromorphic system with high-precision current readout," in *2023 BioCAS*, pp. 1–5, IEEE, 2023.
- [16] C. Ríos, N. Youngblood, Z. Cheng, *et al.*, "In-memory computing on a photonic platform," *Science Adv.*, vol. 5, no. 2, p. eaau5759, 2019.
- [17] H.-T. Lue, K.-C. Wang, and C.-Y. Lu, "Introduction of 3D AND-type flash memory and its applications to computing-in-memory (CIM)," in *2021 VLSI-TSA*, pp. 1–2, IEEE, 2021.
- [18] M. Kim, M. Liu, L. R. Everson, and C. H. Kim, "An embedded nand flash-based compute-in-memory array demonstrated in a standard logic process," *IEEE J. Solid-State Circ.*, vol. 57, no. 2, pp. 625–638, 2021.
- [19] S. Jain, H. Tsai, C.-T. Chen, *et al.*, "A heterogeneous and programmable compute-in-memory accelerator architecture for Analog-AI using dense 2-D mesh," *IEEE Trans. VLSI*, vol. 31, no. 1, pp. 114–127, 2023.
- [20] W. A. Simon, I. Boybat, R. Kodra, *et al.*, "CIMBA: accelerating genome sequencing through on-device basecalling via compute-in-memory," *IEEE Trans. Parallel & Distr. Sys.*, 2025.
- [21] G. W. Burr, H. Tsai, I. Boybat, *et al.*, "Design of Analog-AI hardware accelerators for Transformer-based language models," in *2023 IEDM*, pp. 1–4, IEEE, 2023.
- [22] I. Boybat, T. Boesch, M. Allegra, *et al.*, "Heterogeneous embedded neural processing units utilizing pcm-based analog in-memory computing," in *2024 IEDM*, pp. 1–4, 2024.
- [23] M. Le Gallo, C. Lammie, J. Büchel, *et al.*, "Using the IBM analog in-memory hardware acceleration kit for neural network training and inference," *APL Machine Learning*, vol. 1, no. 4, 2023.
- [24] J. Büchel, W. A. Simon, C. Lammie, *et al.*, "AIHWKIT-lightning: a scalable HW-aware training toolkit for analog in-memory computing," in *NeurIPS 2024 Workshop: Machine Learning with new Compute Paradigms*, 2024.
- [25] M. J. Rasch, C. Mackin, L. Manuel, *et al.*, "Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators," *Nature Comm.*, vol. 14, no. 1, p. 5282, 2023.
- [26] Y. Hou, H. Tsai, K. El Maghraoui, T. Gokmen, G. W. Burr, and L. Liu, "NORA: noise-optimized rescaling of LLMs on analog compute-in-memory accelerators," in *2025 DATE*, IEEE, 2025.
- [27] N. Dey, G. Gosal, H. Khachane, W. Marshall, R. Pathria, M. Tom, J. Hestness, *et al.*, "Cerebras-GPT: Open compute-optimal language models trained on the cerebras wafer-scale cluster," *arXiv:2304.03208*, 2023.
- [28] J. Büchel, A. Vasilopoulos, W. A. Simon, *et al.*, "Efficient scaling of large language models with mixture of experts and 3D analog in-memory computing," *Nature Comput. Sci.*, pp. 1–14, 2025.